

从贝尔实验室诞生的第一个 Unix 系统使用的是 a.out 格式(直到今天,可执行文件仍然称为 a.out 文件)。Windows 使用可移植可执行(Portable Executable, PE)格式。Mac OS-X 使用 Mach-O 格式。现代 x86-64 Linux 和 Unix 系统使用可执行可链接格式(Executable and Linkable Format, ELF)。尽管我们的讨论集中在 ELF 上,但是不管是哪种格式,基本的概念是相似的。

7.4 可重定位目标文件

图 7-3 展示了一个典型的 ELF 可重定位目标文件的格式。ELF 头(ELF header)以一个 16 字节的序列开始,这个序列描述了生成该文件的系统的字的大小和字节顺序。ELF 头剩下的部分包含帮助链接器语法分析和解释目标文件的信息。其中包括 ELF 头的大小、目标文件的类型(如可重定位、可执行或者共享的)、机器类型(如 x86-64)、节头部表(section header table)的文件偏移,以及节头部表中条目的大小和数量。不同节的位置和大小是由节头部表描述的,其中目标文件中每个节都有一个固定大小的条目(entry)。

夹在 ELF 头和节头部表之间的都是节。一个典型的 ELF 可重定位目标文件包含下面几个节:

.text: 已编译程序的机器代码。

.rodata: 只读数据,比如 printf 语句中的格式串和开关语句的跳转表。

.data: 已初始化的全局和静态 C 变量。局部 C 变量在运行时被保存在栈中,既不出现在 .data 节中,也不出现在 .bss 节中。

.bss: 未初始化的全局和静态 C 变量,以及所有被初始化为 0 的全局或静态变量。在目标文件中这个节不占据实际的空间,它仅仅是一个占位符。目标文件格式区分已初始化和未初始化变量是为了空间效率:在目标文件中,未初始化变量不需要占据任何实际的磁盘空间。运行时,在内存中分配这些变量,初始值为 0。

.symtab: 一个符号表,它存放在程序中定义和引用的函数和全局变量的信息。一些程序员错误地认为必须通过 -g 选项来编译一个程序,才能得到符号表信息。实际上,每个可重定位目标文件在 .symtab 中都有一张符号表(除非程序员特意用 STRIP 命令去掉它)。然而,和编译器中的符号表不同,.symtab 符号表不包含局部变量的条目。

.rel.text: 一个 .text 节中位置的列表,当链接器把这个目标文件和其他文件组合时,需要修改这些位置。一般而言,任何调用外部函数或者引用全局变量的指令都需要修改。另一方面,调用本地函数的指令则不需要修改。注意,可执行目标文件中并不需要重定位信息,因此通常省略,除非用户显式地指示链接器包含这些信息。

.rel.data: 被模块引用或定义的所有全局变量的重定位信息。一般而言,任何已初始化的全局变量,如果它的初始值是一个全局变量地址或者外部定义函数的地址,都需要被修改。

.debug: 一个调试符号表,其条目是程序中定义的局部变量和类型定义,程序中定义和引用的全局变量,以及原始的 C 源文件。只有以 -g 选项调用编译器驱动程序时,才

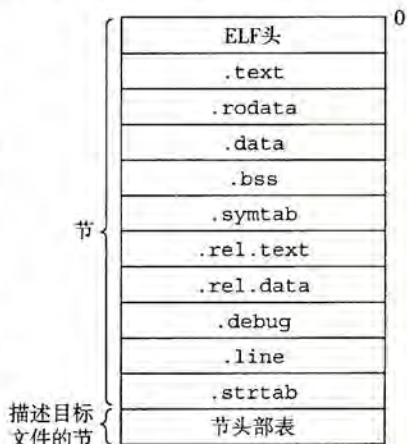


图 7-3 典型的 ELF 可重定位目标文件

会得到这张表。

.line: 原始 C 源程序中的行号和 .text 节中机器指令之间的映射。只有以 -g 选项调用编译器驱动程序时, 才会得到这张表。

.strtab: 一个字符串表, 其内容包括 .symtab 和 .debug 节中的符号表, 以及节头部中的节名字。字符串表就是以 null 结尾的字符串的序列。

旁注 为什么未初始化的数据称为 .bss

用术语 .bss 来表示未初始化的数据是很普遍的。它起始于 IBM 704 汇编语言(大约在 1957 年)中“块存储开始(Block Storage Start)”指令的首字母缩写, 并沿用至今。一种记住 .data 和 .bss 节之间区别的简单方法是把“bss”看成是“更好地节省空间(Better Save Space)”的缩写。

7.5 符号和符号表

每个可重定位目标模块 m 都有一个符号表, 它包含 m 定义和引用的符号的信息。在链接器的上下文中, 有三种不同的符号:

- 由模块 m 定义并能被其他模块引用的全局符号。全局链接器符号对应于非静态的 C 函数和全局变量。
- 由其他模块定义并被模块 m 引用的全局符号。这些符号称为外部符号, 对应于在其他模块中定义的非静态 C 函数和全局变量。
- 只被模块 m 定义和引用的局部符号。它们对应于带 static 属性的 C 函数和全局变量。这些符号在模块 m 中任何位置都可见, 但是不能被其他模块引用。

认识到本地链接器符号和本地程序变量不同是很重要的。.symtab 中的符号表不包含对应于本地非静态程序变量的任何符号。这些符号在运行时在栈中被管理, 链接器对此类符号不感兴趣。

有趣的是, 定义为带有 C static 属性的本地过程变量是不在栈中管理的。相反, 编译器在 .data 或 .bss 中为每个定义分配空间, 并在符号表中创建一个有唯一名字的本地链接器符号。比如, 假设在同一模块中的两个函数各自定义了一个静态局部变量 x :

```

1  int f()
2  {
3      static int x = 0;
4      return x;
5  }
6
7  int g()
8  {
9      static int x = 1;
10     return x;
11 }
```

在这种情况下, 编译器向汇编器输出两个不同名字的局部链接器符号。比如, 它可以用 $x.1$ 表示函数 f 中的定义, 而用 $x.2$ 表示函数 g 中的定义。

给 C 语言初学者 利用 static 属性隐藏变量和函数名字

C 程序员使用 static 属性隐藏模块内部的变量和函数声明, 就像你在 Java 和 C++